

Integer Storage, 1's & 2's Complement, Unicode and UTF Encodings

A concise C++ / interview study guide

The big idea

Keep two topics separate: (1) numbers are stored as binary bit patterns; signed integers use two's complement. (2) text uses Unicode code points, and UTF-8/16/32 define how those code points are encoded into storage.

Topic	Question it answers
Binary / integers	How is a number such as 17 represented in bits?
1's / 2's complement	How are negative signed integers represented?
Unicode	What numeric code point corresponds to a character?
UTF-8 / UTF-16 / UTF-32	How is that Unicode code point encoded into bytes/code units?

1. Bits, Bytes, and Binary Integers

Everything in memory ultimately becomes bits. A bit is 0 or 1, and a byte is typically 8 bits.

1 byte = 8 bits

17 (decimal) = 10001 (binary)
Stored in 8 bits: 00010001

For an unsigned n-bit integer, every bit contributes to the magnitude.

Unsigned n-bit range: 0 to $2^n - 1$

8 bits -> 0 to 255

32 bits -> 0 to 4,294,967,295

Interpretation matters

The bits do not carry a signed/unsigned label themselves. The type and operation determine how the same bit pattern is interpreted.

2. How Negative Integers Are Represented

Three representations are useful to understand historically. Modern signed integer hardware uses two's complement.

2.1 Sign-magnitude

Use the most significant bit as a sign bit: 0 for positive, 1 for negative.

+5 = 00000101
-5 = 10000101

Problem: it creates both +0 and -0.

```
+0 = 00000000
-0 = 10000000
```

2.2 One's complement

To obtain -x, flip every bit of +x.

```
+17 = 00010001
flip = 11101110
So -17 = 11101110 (one's complement)
```

Problem: it still has two zeros.

```
+0 = 00000000
-0 = 11111111
```

2.3 Two's complement

To obtain -x: flip every bit, then add 1.

```
+17      = 00010001
flip     = 11101110
add 1    = 11101111
-17      = 11101111 (two's complement)
```

Why two's complement?

There is only one zero, and ordinary binary addition works naturally for positive and negative values. This greatly simplifies arithmetic hardware.

2.4 Why addition works

```
  0000101    (+5)
+ 11111011   (-5)
-----
1 00000000
```

Discard the carry beyond 8 bits -> 00000000 = 0

2.5 Signed ranges in two's complement

Signed n-bit range: $-2^{(n-1)}$ to $2^{(n-1)} - 1$

8-bit signed -> -128 to 127

32-bit signed -> -2,147,483,648 to 2,147,483,647

8-bit pattern	Signed interpretation	Unsigned interpretation
00000000	0	0
01111111	127	127
10000000	-128	128
11111110	-2	254
11111111	-1	255

Fast way to decode a negative value

If an 8-bit signed pattern starts with 1, invert the bits and add 1 to recover its magnitude, then attach a minus sign. Example: 11101111 -> invert 00010000 -> +1 = 00010001 = 17 -> value is -17.

3. Integer Storage in C++

Exact sizes are implementation-dependent, but the following sizes are common on modern systems.

C++ type	Typical size	Typical signed range / note
char	1 byte	signedness of plain char is implementation-dependent
short	2 bytes	-32,768 to 32,767
int	4 bytes	-2,147,483,648 to 2,147,483,647
long	4 or 8 bytes	platform-dependent
long long	8 bytes	about -9.22e18 to +9.22e18

```
int x = 17;
```

Typical 32-bit representation:
00000000 00000000 00000000 00010001

Important C++ note

Modern C++ (C++20 onward) specifies two's-complement representation for signed integers. For interview reasoning, two's complement is the representation to know.

4. Text Is a Different Problem

Numbers and text should not be mixed conceptually. An integer value such as 17 is not the same thing as the text "17".

C++ value	Meaning	What is stored conceptually
int x = 17;	numeric integer seventeen	binary integer value 17
char c = '1';	character digit one	character code for '1' (U+0031 / decimal 49)
string s = "17";	two text characters	the encoded characters '1' and '7'

```
integer 1    = 00000001 (in 8 bits)
character '1' = 00110001 (U+0031 = decimal 49)
```

5. Unicode: Characters -> Code Points

Unicode is a standardized mapping from characters to numeric code points. A code point is written as U+ followed by a hexadecimal number.

Character	Unicode code point	Decimal value
A	U+0041	65

Character	Unicode code point	Decimal value
a	U+0061	97
é	U+00E9	233
Face-with-tears-of-joy emoji	U+1F602	128514

Mental model

Unicode answers: "Which number represents this character?" It does not by itself say exactly which bytes should appear in a file or string.

6. UTF Encodings: Code Points -> Storage

UTF-8, UTF-16, and UTF-32 are encodings: rules for converting Unicode code points into code units and bytes.

Encoding	Code-unit size	Per-code-point storage	A (U+0041)	U+1F602 emoji
UTF-8	8 bits	1-4 bytes	1 byte	4 bytes
UTF-16	16 bits	1-2 code units	2 bytes	4 bytes
UTF-32	32 bits	1 code unit	4 bytes	4 bytes

6.1 UTF-8

UTF-8 uses 1 to 4 bytes per Unicode code point. ASCII characters occupy one byte, which is a major reason UTF-8 is compact for English-heavy text.

```
A    (U+0041) -> 41          [1 byte]
é    (U+00E9) -> C3 A9       [2 bytes]
€    (U+20AC) -> E2 82 AC    [3 bytes]
U+1F602 emoji -> F0 9F 98 82 [4 bytes]
```

6.2 UTF-16

UTF-16 uses 16-bit code units. Many code points use one code unit (2 bytes); code points outside the Basic Multilingual Plane use a surrogate pair: two 16-bit code units (4 bytes).

```
A    -> 1 UTF-16 code unit = 2 bytes
U+1F602 -> 2 UTF-16 code units = 4 bytes
```

6.3 UTF-32

UTF-32 uses exactly one 32-bit code unit per Unicode code point. It is simple to index by code point, but uses more storage for common text.

```
A    -> 4 bytes
U+1F602 -> 4 bytes
U+4F60    -> 4 bytes
```

```
"HELLO"
UTF-8 -> 5 bytes
UTF-32 -> 20 bytes
```

7. Byte vs Code Point vs Code Unit

Term	Meaning	Example
Byte	Typically 8 bits	11110000
Code point	Unicode number assigned to a character	emoji -> U+1F602
Code unit	Basic storage unit of an encoding	UTF-8: 8 bits; UTF-16: 16 bits; UTF-32: 32 bits

Example: U+1F602 emoji

It is one Unicode code point, but it occupies 4 UTF-8 code units, 2 UTF-16 code units, or 1 UTF-32 code unit.

8. Unicode and C++ String Types

A key correction: `std::string` does not inherently mean UTF-8. It is a sequence of `char` elements (bytes). UTF-8 is commonly stored in `std::string`, but the encoding is a convention chosen by your program/environment.

C++ type	Element / unit	Typical use
<code>char</code>	1 byte	byte / narrow character unit
<code>std::string</code>	sequence of <code>char</code>	often contains UTF-8 bytes
<code>char16_t</code>	16-bit code unit	UTF-16 unit
<code>std::u16string</code>	sequence of <code>char16_t</code>	UTF-16 text
<code>char32_t</code>	32-bit code unit	UTF-32 unit
<code>std::u32string</code>	sequence of <code>char32_t</code>	UTF-32 text
<code>wchar_t</code>	platform-dependent	legacy / platform wide-character APIs

8.1 Why `std::string::size()` can surprise you

```
std::string a = "ABC";
// UTF-8: 3 bytes -> a.size() == 3

std::string b = "<U+1F602 emoji>";
// UTF-8: F0 9F 98 82 -> 4 bytes
// b.size() is typically 4, not 1
```

Interview point

`std::string::size()` counts `char` elements / bytes. It does not count Unicode code points or user-perceived characters.

9. "How Is 17 Stored?" - Resolve the Ambiguity

When someone asks how 17 is stored, first identify whether they mean the numeric integer or text containing the digits 1 and 7.

Expression	Category	Storage idea
<code>int x = 17;</code>	integer	binary number; UTF is irrelevant

Expression	Category	Storage idea
<code>std::string s = "17";</code>	text	two Unicode characters encoded as bytes

```

int x = 17;
32-bit example:
00000000 00000000 00000000 00010001

std::string s = "17";    // UTF-8
'1' = U+0031 -> 0x31
'7' = U+0037 -> 0x37
Total = 2 UTF-8 bytes

```

10. Complete Mental Map

```

DATA
|
+-- Numbers
|   |
|   +-- Unsigned integers -> ordinary binary
|   |
|   +-- Signed integers   -> two's complement
|
+-- Text
|   |
|   +-- Unicode -> character <-> code point
|   |           e.g. emoji -> U+1F602
|   |
|   +-- Encoding -> code point -> storage
|       |
|       +-- UTF-8   : 1-4 bytes
|       +-- UTF-16  : 1-2 x 16-bit units
|       +-- UTF-32  : 1 x 32-bit unit

```

11. Complements - One Table to Memorize

Representation	How to represent -x	Main issue / status
Sign-magnitude	change the sign bit	two zeros; not normal integer representation today
One's complement	flip every bit	two zeros; historical
Two's complement	flip every bit + 1	modern signed integer representation

```

+17 = 00010001

One's complement -17:
11101110

Two's complement -17:
11101111

```

12. Interview Cheat Sheet

Five facts to memorize

1 byte = 8 bits. Unsigned n-bit range = 0 to $2^n - 1$. Signed n-bit range = $-2^{(n-1)}$ to $2^{(n-1)} - 1$. Two's complement = flip bits + 1. Unicode gives code points; UTF defines their encoding.

Concept	Memorize this
Unsigned integer	0 to $2^n - 1$
Signed integer	$-2^{(n-1)}$ to $2^{(n-1)} - 1$
Two's complement	invert bits and add 1
Unicode	character -> code point
UTF-8	1-4 bytes per code point
UTF-16	1-2 x 16-bit code units
UTF-32	1 x 32-bit code unit
std::string	byte / char sequence; often used for UTF-8

Core distinction: integer representation and text encoding are separate topics.